GIG: Graph Data Imputation With Graph Differential Dependencies

Jiang Hua[1], Michael Bewong[2,3], Selasi Kwashie[2], MD Geaur Rahman[3], Junwei Hu[1], Xi Guo[1✉], and Zaiwen Feng[1✉]

[1] College of Informatics, Huazhong Agricultural University, Wuhan, Hubei, China
{zaiwen.feng,xguo}@mail.hzau.edu.cn
[2] AI and Cyber Futures Institute, Charles Sturt University, NSW, Australia
[3] School of Computing, Mathematics and Engineering, Charles Sturt University, NSW, Australia

Abstract. Data imputation addresses the challenge of imputing missing values in database instances, ensuring consistency with the overall semantics of the dataset. Although several heuristics which rely on statistical methods, and ad-hoc rules have been proposed. These do not generalise well and often lack data context. Consequently, they also lack explainability. The existing techniques also mostly focus on the relational data context making them unsuitable for wider application contexts such as in graph data. In this paper, we propose a graph data imputation approach called *GIG* which relies on graph differential dependencies (GDDs). *GIG*, learns the GDDs from a given knowledge graph, and uses these rules to train a transformer model which then predicts the value of missing data within the graph. By leveraging GDDs, *GIG* incoporates semantic knowledge into the data imputation process making it more reliable and explainable. Experimental results on seven real-world datasets highlight GIG's effectiveness compared to existing state-of-the-art approaches.

Keywords: Data Imputation · Transformer · Graph Difference Dependency

1 Introduction

In the era of big data, addressing the challenge of missing value imputation has become critical [1]. In general, datasets with incomplete information pose challenges for deriving reliable knowledge [2]. Consequently, considerable efforts have been directed towards the data imputation problem, viewed as a fundamental task in data cleaning [3]. The complexity of identifying the optimal values for imputing missing data arises from the necessity to evaluate all potential combinations within the value distribution. Numerous approaches in the literature operate under the premise that a missing value can be imputed with another value from the same population, with the overarching goal of maintaining the overall integrity of the data [4]. Furthermore, heuristics have been extensively utilized to judiciously choose values for imputation, aiming to enhance the efficiency of the process without compromising the accuracy of the imputed values [5]. However, most of these techniques focus on relational data scenarios which is not bewildered with the same challenges pertaining to graph data. Thus, the task of addressing data imputation in graph data is a non-trivial task.

This paper presents GIG, a data imputation algorithm for completing missing values in knowledge graphs, based on a Graph Differential Dependency-guided (GDD-guided) *transformer* model. GDDs go beyond Graph Entity Dependencies (GEDs) by integrating distance and matching functions in lieu of equality functions. In general, GDDs exhibit better compatibility with data and are more suitable for discovering and handling various types of errors, such as duplicates, outliers, and constraint violations[6]. Hence, GDDs have the potential to be employed for identifying appropriate candidate values to replace missing ones in the data imputation process. Our proposed technique GIG leverages GDDs to identify instances that satisfy GDD rules to train a transformer model; formulates missing values in the form of rules and leverages the trained transformer to complete the rule; and relies on the GDD rules to validate the semantic consistency of the missing value candidates.

The paper is structured as follows: Section 2 reviews the literature on data imputation approaches. Section 3 introduces preliminary notions on GDDs. Section 4 delineates the data imputation problem. The GIG algorithm is detailed in Section 5, and an experimental evaluation measuring its effectiveness is presented in Section 6. Finally, conclusions and directions for further research are discussed in Section 7.

2 RELATED WORK

In the last decade, various solutions have been proposed for data imputation, including the application of linear regression methods. Specifically, the linear regression model in [7] addresses imputation of numerical missing values to tackle the data sparsity problem, where the number of complete tuples may be insufficient for precise imputation. Further, *REMIAN* proposed in [8] employs a multivariate regression model (MRL) to handle real-time missing value imputation in error-prone environments, dynamically adapting parameters and incrementally updating them with new data. The work in [9] utilizes regression models differently for imputing missing values. Rather than directly inferring missing values, the authors suggest predicting distances between missing and complete values and then imputing values based on these distances. Other techniques such as [10] use regression models to analyse missing values.

In [11], Samad proposes a hybrid framework that combines ensemble learning and deep neural networks (DNN) with the Multiple Imputation using Chained Equations (MICE) approach to improve imputation and classification accuracy for missing values in tabular data [11]. They introduce cluster labels from training data to enhance imputation accuracy and minimize variability in missing value estimates. The paper demonstrates superior performance of their methods compared to baseline MICE and other imputation algorithms, particularly for high missingness and non-random missing types.

The Holoclean framework, introduced in [12] employs machine learning techniques to rectify errors in structured datasets. It addresses inconsistencies like duplicates, incorrect entries, and missing values by automatically generating a probabilistic model. This model extrapolates features representing dataset uncertainty, which are used to characterize a probabilistic graphical model. Statistical learning and probabilistic inference are then applied to repair errors. Additionally, data entry consistency can be maintained by defining integrity constraints. The Holoclean framework has also been shown to be useful in multi-source heterogeneous data [13].

The techniques discussed so far aim to address the data imputation problem by using models that infer relationships among data in a supervised or unsupervised manner. These approaches are notably fast, although with the exception of Holoclean, they are restricted to numerical datasets by nature. In contrast, RENUVER proposed in [14] is suited for numerical, textual, and categorical data, treating each missing value according to the data domain it belongs to. Although the methodology employed in Holoclean can permit the imputation of textual and categorical values with the use of metadata, specifically *denial constraints*, the role of metadata in the imputation process is limited, and primarily serves as integrity constraints that imputed values must satisfy. In contrast, metadata can be more extensively leveraged to generate candidate values, as demonstrated in RENUVER with relational functional dependencies (RFDs).

Indeed, RFDs have been explored in data imputation. Bohannon et al. introduced Conditional Functional Dependencies (CFDs), a specific type of RFDs, capable of capturing overall data correctness [15]. While CFDs capture data semantics and are suitable for cleaning, the authors didn't propose a dedicated imputation approach. They defined SQL queries for CFD violation detection, potentially used for checking imputed dataset integrity. However, determining CFDs involves an expensive, manual process [16, 17]. Another imputation algorithm using RFDs is by Song et al. [18]. They used differential dependencies (DDs) [19], a type of RFDs [20], employing similarity rules for value tolerance. To maximize imputed missing values, the authors presented four algorithms, including integer linear programming, its approximation, a randomized method, and Derand, a derandomized version considered optimal for deterministic approximation of multiple missing values.

3 PRELIMINARIES

In this section we introduce the concept of GDDs and show how GDDs can be used for graph data imputation.

A GDD σ is a pair ($Q[\bar{z}]$, $\Phi_X \rightarrow \Phi_Y$), where: $Q[\bar{z}]$ is a graph pattern called the scope, $\Phi_X \rightarrow \Phi_Y$ is called the dependency, Φ_X and Φ_Y are two (possibly empty) sets of distance constraints on the pattern variables $\bar{z}$. A *distance constraint* in Φ_X and Φ_Y on $\bar{z}$ is one of the following [21]:

$$\begin{aligned} d_A(x.A, c) \leq t_A;\ & d_{A_iA_j}(x.A_i, x'.A_j) \leq t_{A_iA_j}; \\ d_{\equiv}(x.eid, C_e) = 0; \quad & d_{\equiv}(x.eid, x'.eid) = 0; \\ d_{\equiv}(x.rela, C_r) = 0; \quad & d_{\equiv}(x.rela, x'.rela) = 0; \end{aligned}$$

where $x, x' \in \bar{z}$, A, A_i, A_j are attributes in $\mathbf{A}$, c is a value of A, $d_{A_iA_j}(x.A_i, x'.A_j)$ (or $d_{A_i}(x, x')$ if $A_i = A_j$) is a user specified distance function for values of (A_i, A_j), $t_{A_iA_j}$ is a threshold for (A_i, A_j), $d_{\equiv}(\cdot,\cdot)$ are functions on **eid** and relations and they return 0 or 1. $d_{\equiv}(x.eid, C_e) = 0$ if the eid value of x is C_e, $d_{\equiv}(x.eid, x'.eid) = 0$ if both x and x' have the same eid value, $d_{\equiv}(x.rela, C_r) = 0$ if x has a relation named rela and ended with the profile/node C_r, $d_{\equiv}(x.rela, x'.rela) = 0$ if both x and x' have the relation named rela and ended with the same profile/node.

The user-specified distance function $d_{A_iA_j}(x.A_i, x^{'}.A_j)$ depends on the types of A_i and A_j. It can take the form of an arithmetic operation involving interval values, an edit distance calculation for string values, or the distance computation between two categorical values within a taxonomy, among other possibilities. These functions accommodate the wildcard value '*' to represent any domain, and in such cases, they return a 0 distance.

We call Φ_X and Φ_Y the LHS and the RHS functions of the dependency respectively. In this work we rely on the GDDMiner proposed in [22] to mine the GDD rules from graph data.

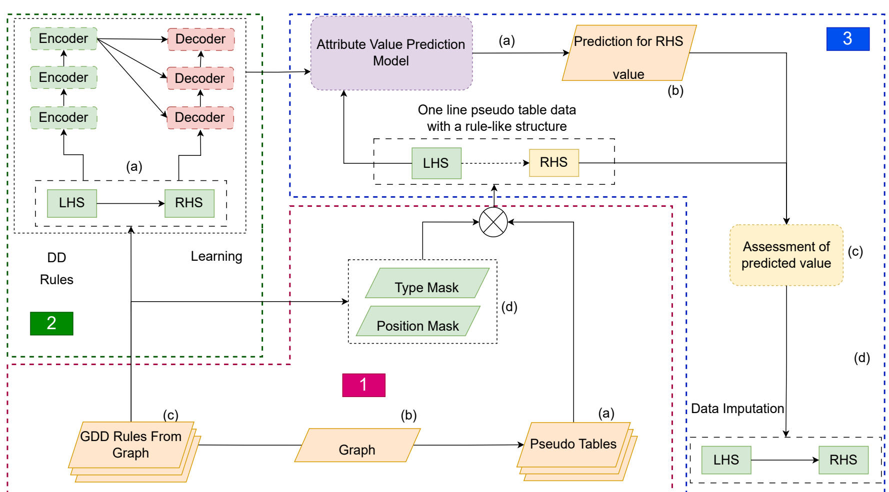

Fig. 1: *GIG* Framework

4 *GIG* ALGORITHM

GIG relies on a transformer architecture and graph differential dependencies (GDDs) for data imputation. Given a knowledge graph G, GIG proceeds in three steps namely (1) rule mining and selection; (2) transformer training; and (3) missing value imputation. These steps are summarised in Figure 1 and explained as follows.

4.1 Rule Mining and Selection

The GDD mining algorithm proposed in [22] is employed to extract GDD rules from the knowledge graph G as shown in Figure 1, Part 1. For example $d(y.name, \hat{y}.name) \leq 1 \rightarrow d(y.genre, \hat{y}.genre) = 0$ is one such rule that implies any two entities y and $\hat{y}$ with a similar name will belong to the same genre.

GDD rules are composed of a left-hand side (LHS) and a right-hand side (RHS). The LHS serves as the input to the transformer encoder, while the RHS serves as the input to the transformer decoder (*cf.* Figure 1, Part 2 (a)). When dealing with multiple GDD rules, where two or more rules exhibit distinct left-hand sides (LHS) but share the same right-hand side (RHS), we opt to consolidate the LHS to enhance the predictive outcomes of the transformer model. Similarly, in cases where multiple rules feature the same LHS but different RHS, we amalgamate the RHS. For example, a rule set $X \rightarrow Y_1$, $X \rightarrow Y_2$, $X \rightarrow Y_3$, will be merged to $X \rightarrow Y_1, Y_2, Y_3$. This adaptation is implemented to consolidate the rules and streamline the learning process of the transformer model.

In the case of GDD rules, we employ masking vectors for the storage of attribute and type details. Both the left-hand side (LHS) and right-hand side (RHS) of GDD rules encompass attribute information, and the utilization of masking vectors for this purpose is both succinct and efficient. For instance, consider a tuple h1 from Table 1. Let us assume we have a GDD rule $r : x.name \rightarrow y.genre$, in this case, we can construct a mask for the attributes, represented by $(0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0)$, where a value of 1 indicates that the attribute is utilized. This approach enables us to preserve a substantial amount of information related to GDD rules.

Table 1: pseudo-relational table

Pattern variables	x		y					y'			
Attributes / Matches	id	Name (A1)	id	Name (A2)	Genre (A3)	Year (A4)	Price (A5)	id	Name (A2)	Genre (A3)	Year (A4)
h1	1	GL	4	AF9	Racing	2018	£50	5	AF11	Racing	2018
h2	3	EA	7	?	Soccer	2019	£55	8	F20	Soccer	2019
h3	3	EA	7	F20	Soccer	2019	£55	9	F21	Soccer	2020
h4	3	EA	8	F20	Soccer	2019	£60	9	F21	Soccer	2020

4.2 Transformer Training

The transformer consists of an encoder and a decoder (*cf.* Figure 1, Part 2), where the LHS of the rules serves as the input to the encoder, and the RHS as the input to the decoder. Thus, for a GDD rule, $(Q, X \to Y)$, the transformer's encoder learns the features of X and the decoder learns the features of Y. In this way, the transformer learns the relationship between the LHS and RHS of the rules.

Encoder. Given a rule $(Q, X \to Y)$, X contains one or more distinct attribute values, which are all considered as a single unit during the training phase. For example, the LHS $X := d(y.name, y'.name) \leq 1$, will be treated as a literal $\{y.name, y'.name, \leq 1\}$. By treating $(y.name, y'.name)$ and ≤ 1 as unified literal within the encoding framework, we enable the transformer to infer their interconnectedness. We have observed, empirically, that this approach is more robust against the potential of variations within the attribute values which can deteriorate model performance.

Decoder. The decoder uses a similar approach to the encoder. When handling the RHS $Y := d(y.gender, y'.gender) = 0$, by treating $\{y.gender, y'.gender, = 0\}$ as unified literal.

Training. Once the inputs to the encoder (LHS) and decoder (RHS) are derived from the rules, the transfomer model learns GDD rules by minimising the loss function *i.e.* Kullback-Leibler Divergence (KLDivLoss). KLDivLoss measures the divergence between the predicted distribution and the target distribution. Lower values of the KLDivLoss indicate better alignment between input and output, which is desirable during training. The final result after training serves as the predictive model.

4.3 Imputing missing value

Imputing missing value. Missing values can also be expressed as rules for example $X \to \_$ implies the LHS values X are known and the values of Y are to be determined. GIG uses the LHS in the trained transformer predictive model to predict an RHS value for imputing Y, facilitating the identification of all viable candidates for the imputation.

Let us consider a scenario where we are interested in predicting the y.name value of record h2 in the pseudo-relational table Table 1. The first step is to identify all GDD rules that has y.name in its RHS. That is any rule that has a '1' in its positional mask at index 3 is considered. For example, the rule $x.name \to y.name$ which has the positional mask (0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0) will be considered since it has a value '1' in the index position 3. Intuitively, the rule means any two nodes of type x that share the same name, will also share the same name on a node type y.

Next, based on the rules identified to be relevant, we can identify the LHS value which should serve as input to the transformer model. In this case EA becomes the input since according to the rule, the LHS should consists of x.name, and in h2 x.name is EA. The transformer then makes a prediction of what the

RHS value should be. The predicted value can either be *semantically consistent* or *inconsistent*. For instance, consider the scenario where the transformer predicts a value of "2020". In this scenario, when this value is imputed, it violates the rule $x.name \rightarrow \ y.name$ since "2020" is not a *name* attribute but rather a *year* attribute. Thus this value is discarded. However, if the transformer model predicts the value "F20", this value can be accepted as an imputed value since it does not violate the rule $x.name \rightarrow \ y.name$. It is possible that there can be several matching rules and several predicted candidates, for simplicity in this work, we only rely on the top 1 rule and predicted candidates for imputation.

The GIG algorithm is summarised in Algorithm 1 [4].

Algorithm 1 GIG

Ensure: Graph differential dependencies $GDDs$, pseudo table T and knowledge graph G with missing values
Require: Imputed knowledge graph G'

Construct encoder input W_l, decoder output W_r, and mask B from $GDDs$
for each epoch **do**
  for each batch $\in W_l$ and W_r **do**
    Get encoder input and decoder output
    Perform forward pass through encoder and decoder; Compute loss
    Perform backward pass and parameter update
  end for
end for
$\mathcal{M} \xleftarrow{save}$ transformer.model
for $t \in T$ **do**
  if t has missing value **then**
    $M \leftarrow$ index position(s) of missing value(s)
    for $b \in B$ **do**
      if $b.M \neq 0$ **then**
        Get rule(s) $X \rightarrow Y$ from b such that Y contains $b.M$
        $I \leftarrow$ index position(s) of $X.attributes$
        return
      end if
    end for
    $t'.M \xleftarrow{predicted} \mathcal{M}(\{t.I\})$
    Impute G' with prediction $t'.M$
  end if
end for
return G'

The worst-case time complexity of GIG is $O(n \cdot m \cdot \Sigma)$, where n represents the number of GDD rules, m represents the number of rows in the pseudo-relational table, and Σ represents the time taken for one run of the transformer. To elabo-

[4] All source codes and datasets used in this paper will be made publicly available upon acceptance of the paper

rate further, each row in the pseudo-relational table is compared to each rule in the worst case when finding matching rules for a missing value, the time complexity is $O(n \cdot m)$. Next, the time complexity of running the transformer model on a single output from the GDD rule and pseudo relational table comparison is Σ. Combining both steps, the overall worst-case time complexity of GIG is $O(n \cdot m \cdot \Sigma)$.

5 Experiment

5.1 Experimental Setting

All experiments were performed on a single server, Intel(R) Xeon(R) Silver 4210R CPU @ 2.40GHz, 32GB RAM. We also adopt *transformer* architecture following the work in [23].

Datasets. In this paper, we use seven real world datasets summarised in Table 2 for evaluation. Namely (1) *Restaurant* which includes six attributes, including the name and address of the restaurant; (2) *Customer_shopping_data (CSD)* which contains six attributes, including the customer's age and the product price; (3) *Adult* which has 11 attributes, including an adult's occupation and nationality; (4) *Ncvoter* which has six attributes, including the patient's age and registration date; (5) *Entity Resolution*, a dataset derived from a virtual online video streaming platform; (6) *Women's World Cup* which contains five node types representing the *person*, *team*, *squad*, *tournament* and *match* from all the World Cups between 1991 and 2019; (7) *Graph Data Science* dataset which reflects the connections between different airports around the world.

For all datasets, we vary the percentage of missing values in the range [1%, 5%] on each considered dataset, with random selection of the data values. We note that, *Restaurant*, *Customer_shopping_data*, *Adult* and *Ncvoter* summarised in Table 1, are relational data that have also been used in benchmark techniques [9, 12]. For these datasets, we first convert them into knowledge graphs, from which we extract GDD rules and generate pseudo-relational tables [22].

Evaluation Baselines In this work, we rely on Holoclean [5] and DERAND [9] as baselines for comparison. DERAND employs tolerance-based similarity rules instead of strict editing rules to discard invalid candidates identified by similarity neighbors. Holoclean is a Holistic machine-learning solution for data imputation [5].

Evaluation metrics. The effectiveness of the data imputation approaches was evaluated using three different metrics: precision, recall, and F1-measure, which is consistent with the existing literature [14].

Table 2: Summary of real-world datasets

Dataset	# **Attributes**	# **Tuples/Nodes**	# **missing values**				
			[1%]	**[2%]**	**[3%]**	**[4%]**	**[5%]**
Restaurant	6	864	52	104	155	206	259
Customer_shopping_data	6	500	45	90	135	180	225
Adult	11	500	55	110	165	220	275
Ncvoter	7	500	35	70	105	140	175
Entity Resolution	13	676	88	174	261	348	435
Graph data Science	7	519	36	72	108	144	180
WomensWorldCup2019	13	514	66	132	198	254	330

In this context, precision quantifies how many missing values are correctly imputed relative to the total number of imputed missing values. This parameter serves as a reliability score, indicating how well the algorithm performs when deciding whether to impute with an uncertain value or leave the missing value. That is, if "true" represents the correctly imputed missing values and "imputed" represents all the imputed missing values, then the precision is calculated as follows: precision $= \frac{|\text{true} \cap \text{imputed}|}{|\text{imputed}|}$.

Recall represents the fraction of correctly imputed missing values. That is, if "missing" represents the missing values in the dataset and "true" represents the correctly imputed missing values at the end of the imputation process, then the recall is calculated as follows: recall $= \frac{|\text{true} \cap \text{missing}|}{|\text{missing}|}$.

The F1-measure is computed by combining precision and recall according to the following formula: F1-measure $= \frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$.

5.2 Effectiveness on Relational Data

In this section we compared our proposed method GIG with known relational data based techniques such as Derand [9] and Holoclean [12]. Figure 2 are the results. In Figure 2 we observe that GIG performs competitively with Holoclean and Derand. In particular, across all datasets, GIG has a higher precision rate than other techniques due to the rule guided transformer approach. We do notice, however that with the exception of *Restaurant* dataset, the rule guided approach also leads to a comparatively lower recall on *Adult*,*CSD* and *Ncvoter* datasets. Yet still, GIG is highly competitive and in most cases has a better performance in F1 across all the datasets.

5.3 Effectiveness on Graph Datasets:

We assess the effectiveness of GIG for imputing missing values in graph data. We evaluate GIG's performance on the three graph datasets *Entity Resolution*, *Graph Data Science*, and *WomensWorldCup2019*. Figure 3 is the result of that experiment. In the figure, we observe that the results for *Entity Resolution* were the best, as its data distribution yields more exemplar points that enables the

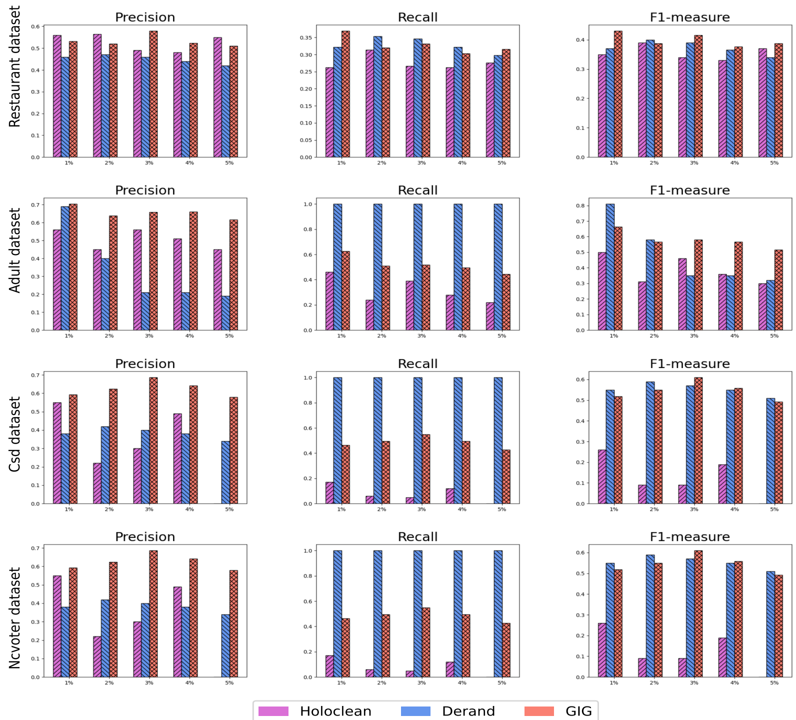

Fig. 2: Comparison of GIG with Derand and Holoclean

GDD-guided transformer model to represent the data more effectively. This is in contrast with the other two datasets.

6 Conclusion

In this paper, we introduced GIG, a data imputation algorithm that leverages graph differential dependencies for missing value imputation. GIG relies on a rule-guided transformer architecture and has three main steps involving GDD rule mining, transformer training and missing value imputation. Evaluation results from seven datasets and in comparison with existing state-of-the-art demonstrate the competitiveness and suitability of GIG for both relational and graph data imputation. In our future work, we aim to enhance GIG by considering different types of rules such as approximate graph entity rules which may improve the overall recall of GIG.

7 Acknowledgement.

This research project was supported in part by National Key Research and Development Program of China under Grant 2023YFF1000100, and in part by

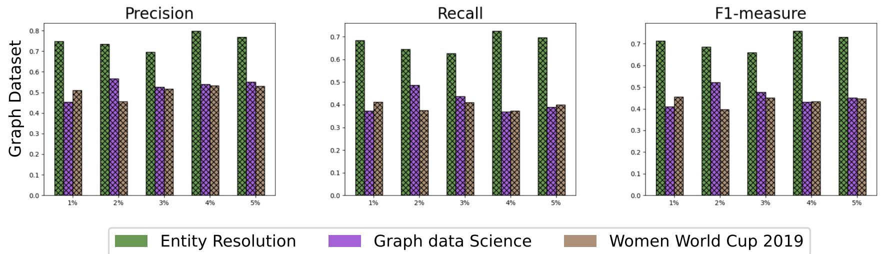

Fig. 3: Performance of GIG on Graph Datasets

the Fundamental Research Funds for the Chinese Central Universities under Grant 2662023XXPY004.

REFERENCES

[1] Gustavo EAPA Batista and Maria Carolina Monard. "An analysis of four missing data treatment methods for supervised learning". In: *Applied artificial intelligence* 17.5-6 (2003), pp. 519–533.

[2] Besay Montesdeoca, Julián Luengo, Jesús Maillo, Diego García-Gil, Salvador García, and Francisco Herrera. "A First Approach on Big Data Missing Values Imputation." In: *IoTBDS*. 2019, pp. 315–323.

[3] Xu Chu, Ihab F Ilyas, Sanjay Krishnan, and Jiannan Wang. "Data cleaning: Overview and emerging challenges". In: *Proceedings of the 2016 international conference on management of data*. 2016, pp. 2201–2206.

[4] A Rogier T Donders, Geert JMG Van Der Heijden, Theo Stijnen, and Karel GM Moons. "A gentle introduction to imputation of missing values". In: *Journal of clinical epidemiology* 59.10 (2006), pp. 1087–1091.

[5] Pinar Cihan and Zeynep Banu Ozger. "A new heuristic approach for treating missing value: ABCIMP". In: *Elektronika ir Elektrotechnika* 25.6 (2019), pp. 48–54.

[6] Ihab F Ilyas and Xu Chu. "Trends in cleaning relational data: Consistency and deduplication". In: *Foundations and Trends® in Databases* 5.4 (2015), pp. 281–393.

[7] Aoqian Zhang, Shaoxu Song, Yu Sun, and Jianmin Wang. "Learning individual models for imputation". In: *2019 IEEE 35th International Conference on Data Engineering (ICDE)*. IEEE. 2019, pp. 160–171.

[8] Qian Ma, Yu Gu, Wang-Chien Lee, Ge Yu, Hongbo Liu, and Xindong Wu. "REMIAN: Real-time and error-tolerant missing value imputation". In: *ACM Transactions on Knowledge Discovery from Data (TKDD)* 14.6 (2020), pp. 1–38.

[9] Shaoxu Song, Yu Sun, Aoqian Zhang, Lei Chen, and Jianmin Wang. "Enriching data imputation under similarity rule constraints". In: *IEEE transactions on knowledge and data engineering* 32.2 (2018), pp. 275–287.

[10] Carolina Gil Marcelino, Gabriel MC Leite, P Celes, and Carlos Eduardo Pedreira. "Missing data analysis in regression". In: *Applied Artificial Intelligence* 36.1 (2022), p. 2032925.

[11] Manar D Samad, Sakib Abrar, and Norou Diawara. "Missing value estimation using clustering and deep learning within multiple imputation framework". In: *Knowledge-based systems* 249 (2022), p. 108968.

[12] Theodoros Rekatsinas, Xu Chu, Ihab F Ilyas, and Christopher Ré. "Holoclean: Holistic data repairs with probabilistic inference". In: *arXiv preprint arXiv:1702.00820* (2017).

[13] Qin Cui, Wenkui Zheng, Wei Hou, Ming Sheng, Peng Ren, Wang Chang, and XiangYang Li. "HoloCleanX: A Multi-source Heterogeneous Data Cleaning Solution Based on Lakehouse". In: *International Conference on Health Information Science*. Springer. 2022, pp. 165–176.

[14] Bernardo Breve, Loredana Caruccio, Vincenzo Deufemia, and Giuseppe Polese. "RENUVER: A Missing Value Imputation Algorithm based on Relaxed Functional Dependencies." In: *EDBT*. 2022, pp. 1–52.

[15] Qi Wang, Pang-Ning Tan, and Jiayu Zhou. "Imputing structured missing values in spatial data with clustered adversarial matrix factorization". In: *2018 IEEE International Conference on Data Mining (ICDM)*. IEEE. 2018, pp. 1284–1289.

[16] Wenfei Fan, Floris Geerts, Jianzhong Li, and Ming Xiong. "Discovering conditional functional dependencies". In: *IEEE Transactions on Knowledge and Data Engineering* 23.5 (2010), pp. 683–698.

[17] Samir Al-Janabi and Ryszard Janicki. "Data repair of density-based data cleaning approach using conditional functional dependencies". In: *Data Technologies and Applications* 56.3 (2022), pp. 429–446.

[18] Shaoxu Song and Lei Chen. "Differential dependencies: Reasoning and discovery". In: *ACM Transactions on Database Systems (TODS)* 36.3 (2011), pp. 1–41.

[19] Shaoxu Song and Lei Chen. *Integrity Constraints on Rich Data Types*. Springer Nature, 2023.

[20] Tobias Bleifuß, Sebastian Kruse, and Felix Naumann. "Efficient denial constraint discovery with hydra". In: *Proceedings of the VLDB Endowment* 11.3 (2017), pp. 311–323.

[21] Selasi Kwashie, Lin Liu, Jixue Liu, Markus Stumptner, Jiuyong Li, and Lujing Yang. "Certus: An effective entity resolution approach with graph differential dependencies (GDDs)". In: *Proceedings of the VLDB Endowment* 12.6 (2019), pp. 653–666.

[22] Yidi Zhang, Selasi Kwashie, Michael Bewong, Junwei Hu, Arash Mahboubi, Xi Guo, and Zaiwen Feng. "Discovering Graph Differential Dependencies". In: *Australasian Database Conference*. Springer. 2023, pp. 259–272.

[23] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. "Attention is all you need". In: *Advances in neural information processing systems* 30 (2017).